

DAEDALUS

Vision Document v1.0 — The Labyrinth That Builds Itself

A Mathematically Rich, Plugin-Driven Graph Engine for the Modern Era

Executive Summary

Daedalus is not a maze game. It is a foundational graph and procedural generation engine with an exceptionally deep mathematical core, a production-grade plugin system, and near-unlimited extensibility.

While it currently manifests as a maze generation and solving platform, its true nature is that of a **general-purpose network topology and intelligent routing engine** — one that can power everything from load balancers and service meshes to chaos engineering platforms and educational tools.

Core Thesis: The same algorithms that generate beautiful, mathematically perfect mazes can generate realistic, locality-aware, fault-tolerant network topologies — and the same solvers that find paths through mazes can find optimal routes through those networks.

Vision (2026–2029)

By 2029, Daedalus aims to become the standard open-source engine for procedural graph generation and intelligent routing. We want it to be the go-to library when someone needs realistic network topologies, the default choice for intelligent routing, and a thriving plugin ecosystem with 100+ community contributions.

Positioning

"Daedalus is to graph algorithms what Unity is to game development — a powerful, extensible engine with deep mathematical roots and a vibrant ecosystem."

Strategic Use Cases

1. Load Balancing & Service Mesh (Highest Priority)

Daedalus can become the topology and routing brain for modern load balancers and service meshes. Use Hilbert, Gauss, or Kraken generators to create realistic topologies. Use A*, Bidirectional, or Dead-End Filling solvers as advanced load balancing strategies. Use Turing and Kraken generators for chaos engineering and resilience testing.

2. Network Simulation & Research

Academic research, data center network design, CDN edge placement, and blockchain sharding topology design.

3. Education & Visualization

The most beautiful way to teach graph algorithms with interactive Hilbert curve + A* demonstrations.

Deep Dive: Hilbert Curve + Graph Algorithms in Networks

The **Hilbert Curve** has unique mathematical properties that make it exceptionally valuable for network topology:

Property	Benefit for Networks
Excellent Locality	Nearby cells in 1D are usually nearby in 2D
No Sudden Jumps	Unlike Morton/Z-order, never jumps across large distances
Recursive Structure	Natural hierarchy — perfect for hierarchical routing
Space-Filling	Visits every node exactly once while preserving adjacency
Fractal Nature	Self-similar at every scale — great for multi-level networks

Concrete Ideas for LoadBalancer Lab

- **Hilbert-Based Consistent Hashing:** Map service instances onto a Hilbert curve for better locality and failure isolation.
- **Hilbert + A* for Latency-Aware Routing:** Generate Hilbert topology, assign real latency weights, then use A* for lowest-latency paths.
- **Multi-Level Hilbert Routing:** Coarse curves across regions → finer curves inside data centers → natural hierarchical routing.
- **Hilbert + Kraken for Chaos-Resilient Topologies:** Combine locality with organic, chaotic branching for high redundancy under failure.

Technical Roadmap

Phase	Timeline	Key Milestones
Foundation	Now – Q3 2026	Public GitHub release, web visualization, REST/GraphQL API
Ecosystem	Q4 2026 – Q2 2027	Plugin marketplace, load balancer integrations, academic papers
Platform	2027–2028	Daedalus Cloud, visual designer, AI-assisted generator creation

Daedalus is not finished. It is just getting started.

This engine has the potential to become one of the most respected open-source projects in procedural generation and graph algorithms — with real impact on load balancing, networking, education, and research.